



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Name of Student: Sumit Kr. Mahato

PRN: 25070122195

Subject: Programming with Java

Sem: III AY: 2026-27

Assignment No: 6

TITLE: Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

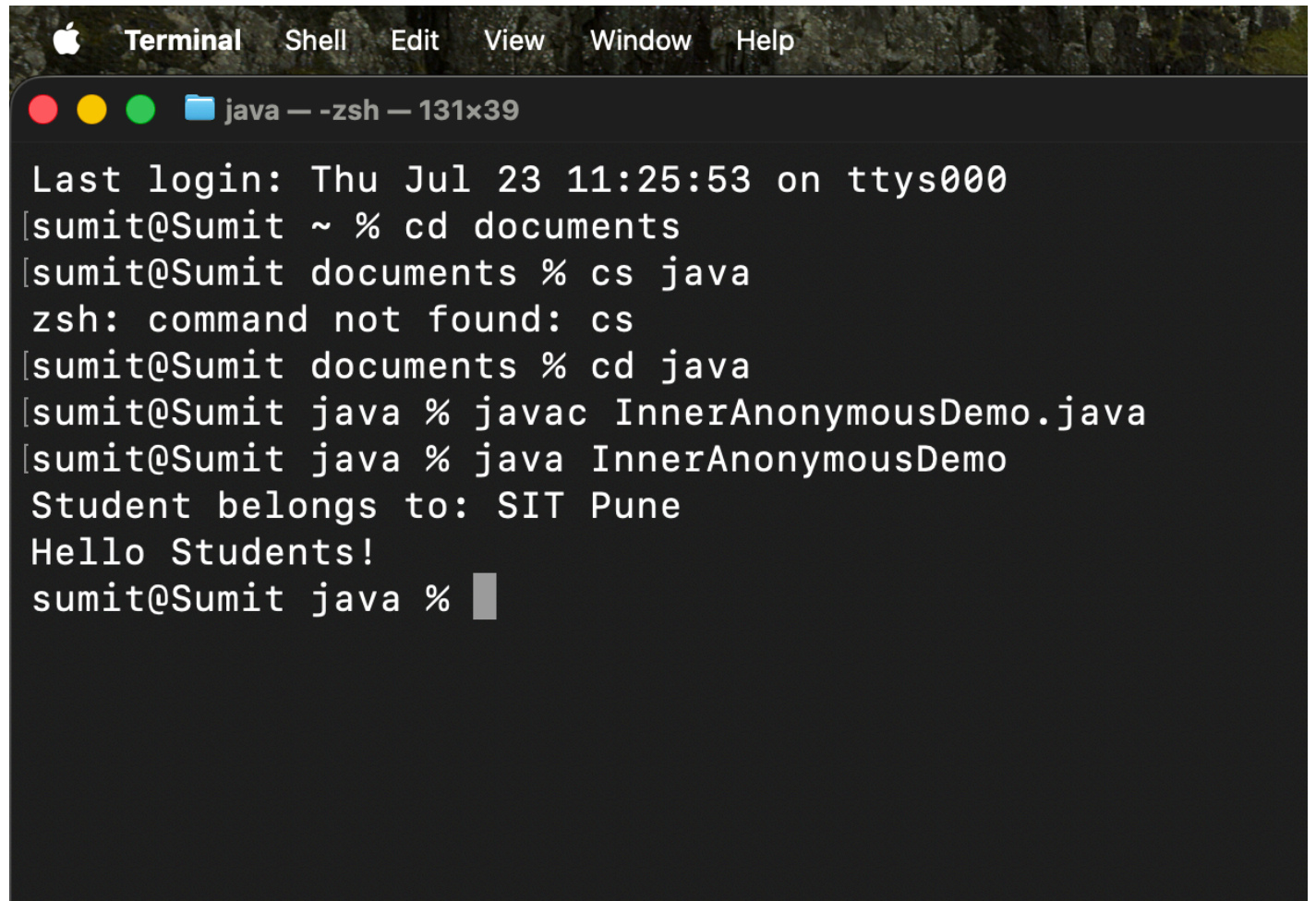
Theory

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling.

Example code:

```
leet.c leet2.c InnerAnonymousDemo.java 1 X
Users > sumit > Documents > java > InnerAnonymousDemo.java > Greeting
1  class College {
2      String collegeName = "SIT Pune";
3
4      // Inner Class
5      class Student {
6          void display() {
7              System.out.println("Student belongs to: " + collegeName);
8          }
9      }
10 }
11
12 interface Greeting {
13     void sayHello();
14 }
15
16 public class InnerAnonymousDemo {
17     Run | Debug
18     public static void main(String[] args) {
19
20         College college = new College();
21         College.Student student = college.new Student();
22         student.display();
23
24         Greeting g = new Greeting() {
25             public void sayHello() {
26                 System.out.println(x: "Hello Students!");
27             }
28         };
29
30         g.sayHello();
31     }
```

Output:

A screenshot of a macOS Terminal window. The title bar at the top shows the Apple logo, the word "Terminal", and menu items "Shell", "Edit", "View", "Window", and "Help". Below the title bar is a dark gray bar with three colored window control buttons (red, yellow, green) and a folder icon followed by the text "java — -zsh — 131x39". The main area of the terminal is black with white text. The text shows a user logging in, navigating to the 'documents' directory, attempting to run 'cs java' (which fails with 'command not found'), then navigating to the 'java' directory and successfully compiling and running 'InnerAnonymousDemo.java'. The program outputs "Student belongs to: SIT Pune" and "Hello Students!". The prompt "sumit@Sumit java %" is followed by a gray cursor bar.

```
Terminal  Shell  Edit  View  Window  Help
java — -zsh — 131x39

Last login: Thu Jul 23 11:25:53 on ttys000
[sumit@Sumit ~ % cd documents
[sumit@Sumit documents % cs java
zsh: command not found: cs
[sumit@Sumit documents % cd java
[sumit@Sumit java % javac InnerAnonymousDemo.java
[sumit@Sumit java % java InnerAnonymousDemo
Student belongs to: SIT Pune
Hello Students!
sumit@Sumit java %
```

Exercises:

1. Create a Vehicle program where an inner class displays vehicle details and anonymous class performs an action.

Source Code:

```
Users > sumit > Documents > java > InnerAnonymousDemo.java > Vehicle > vehicleName
1  class Vehicle {
2      String vehicleName = "Toyota";
3      int speed = 100;
4
5      // Inner Class
6      class Details {
7          void display() {
8              System.out.println("Vehicle Name: " + vehicleName);
9              System.out.println("Vehicle Speed: " + speed + " Km/hr");
10         }
11     }
12 }
13
14 interface Action {
15     void perform();
16 }
17
18 public class InnerAnonymousDemo {
19     Run | Debug
20     public static void main(String[] args) {
21
22         Vehicle vehicle = new Vehicle();
23         Vehicle.Details details = vehicle.new Details();
24         details.display();
25
26         Action a = new Action() {
27             public void perform() {
28                 System.out.println("Vehicle is running.");
29             }
30         };
31
32         a.perform();
33     }
```

OUTPUT:

```
sumit@Sumit java % javac InnerAnonymousDemo.java
sumit@Sumit java % java InnerAnonymousDemo
Vehicle Name: Toyota
Vehicle Speed: 100 Km/hr
Vehicle is running.
sumit@Sumit java %
```

2. Develop a Food Delivery Application where an inner class handles order details and anonymous classes handle delivery status updates.

Source Code:

```
Users > sumit > Documents > java > java1 > InnerAnonymousDemo.java > InnerAno
1  class FoodDelivery
2  {
3      String foodItem = "Pizza";
4      int quantity = 2;
5
6      // Inner Class
7      class Order
8      {
9          void display()
10         {
11             System.out.println("Food Item: " + foodItem);
12             System.out.println("Quantity: " + quantity);
13         }
14     }
15 }
16
17 interface Delivery
18 {
19     void updateStatus();
20 }
21
```

```

21
22 public class InnerAnonymousDemo
23 {
24     public static void main(String[] args)
25     {
26         FoodDelivery food = new FoodDelivery();
27         FoodDelivery.Order order = food.new Order();
28         order.display();
29
30         Delivery d = new Delivery()
31         {
32             public void updateStatus()
33             {
34                 System.out.println("Order is out for delivery.");
35             }
36         };
37
38         d.updateStatus();
39     }
40 }

```

Output:

```

java1 — -zsh — 145x43
[sumit@Sumit java1 % javac InnerAnonymousDemo.java
[sumit@Sumit java1 % java InnerAnonymousDemo
Food Item: Pizza
Quantity: 2
Order is out for delivery.
sumit@Sumit java1 %

```

Conclusion:

This experiment demonstrated the use of inner classes and anonymous classes in Java. Inner classes help organize related code and provide direct access to the members of the enclosing class, improving encapsulation. Anonymous classes allow one-time implementation of interfaces or abstract classes without creating separate class files, making the code simpler and more concise. Overall, these concepts improve code organization, readability, and flexibility, and are widely used in Java applications for tasks such as event handling and GUI development.